Custom Keyboard Layout Generator

Noah Klein
Department of Computer Science
Colorado State University
Fort Collins, CO, USA
klein77@colostate.edu

Trent Nosky
Department of Computer Science
Colorado State University
Fort Collins, CO, USA
tnosky@colostate.edu

***Abstract*—Standard keyboard layouts like QWERTY prioritize mechanical constraints over human ergonomics, often leading to inefficient typing patterns and increased physical strain. While alternative layouts such as Dvorak and Colemak exist, they are designed for the "average" user rather than adapting to individuals. This project introduces ErgoTyper, an embedded machine learning system that optimizes keyboard layouts based on real-time hand tracking. We developed an Android application that utilizes a quantized YOLOv8 neural network to track hand positions on the edge, correlating spatial data with keypress events. This data feeds into a optimization algorithm that evolves a personalized keyboard layout by minimizing aggregate finger travel distance. Our experiments demonstrate that the embedded system successfully runs at 30 FPS on a standard mobile device and generates layouts that reduce finger travel distance by approximately 42% compared to QWERTY for standard English text. The system provides a concept for personalized ergonomic tools.**

I. Introduction

For the past 150 years, the QWERTY layout has dominated modern typing systems. Originally engineered by Christopher Latham Sholes in the 1870s for mechanical typewriters, QWERTY intentionally spaced frequently paired letter combinations apart to prevent jamming. Sholes filed his patent application for this earliest writing machine in October 1867. While effective for the mechanical limitations of the 19th century, this layout is inefficient for modern digital typing.

The consequences of this legacy design are not just inefficiency, but physical injury. Repetitive Strain Injury (RSI) and Carpal Tunnel Syndrome affect millions of computer users annually. These conditions are often exacerbated by the high frequency of ulnar deviation (twisting the wrist outwards) and excessive finger travel required by QWERTY.

Alternative layouts such as Dvorak, Colemak, and Workman aim to minimize overall finger movement by placing more commonly used keys on the home row or favoring lateral ergonomic efficiency. Table I compares the distribution of keystrokes by row across several major layouts.

TABLE I
KEYSTROKE DISTRIBUTION BY KEYBOARD ROW

Row	QWERTY	Dvorak	Colemak
Top Row	49.8%	23.3%	16.2%
Home Row	33.9%	68.7%	72.4%
Bottom Row	17.4%	7.9%	11.4%

This data demonstrates the inefficiency of QWERTY, which relies heavily on top-row movements. Our goal is to design a method for collecting real-time hand motion data to measure the actual ergonomic cost of reaching specific keys for individual users.

This project tackles the one-size-fits-all problem by building a system that measures how you specifically type. The core idea is simple: measure the physical cost of hitting every key for a specific user, then use that data to build a better keyboard for them.

Our approach has two main parts: 1. Data Collection (Edge AI) An Android app that uses computer vision to watch your hands in real-time. It logs exactly where your hands are when you press a key. This moves the analysis from a fancy lab to your own phone. 2. Layout Optimization (Algorithmic) A genetic algorithm that takes that strain data and evolves a new layout optimized for a big chunk of English text. It treats layout design as a math problem, balancing how often a letter is used with how hard it is to reach.

The main contributions here are:

- A way to collect ergonomic data using just a regular phone camera.
- A real-time YOLOv8 object detection pipeline running on Android.
- A new optimization framework that mixes physical tracking data with language stats to make personalized layouts.

II. Related Work

Keyboard optimization isn't new, but most of it has been static analysis.

A. Static Layout Optimization

Several layouts have attempted to dethrone QWERTY, but they remain niche.

- **Dvorak (1936):** Designed to be faster and more ergonomic by placing the most common keys (AOEUIDHTNS) on the home row [2]. This drastically reduces finger movement and encourages alternating hands, which is rhythmically more efficient.
- **Colemak (2006):** This layout aims to reduce finger travel while keeping the learning curve manageable. It keeps popular shortcuts (Ctrl+Z/X/C/V) in their QWERTY positions but optimizes the rest. It places punctuation in more accessible places than Dvorak.

- **Workman:** This layout specifically targets lateral movement. It acknowledges that extending the index finger sideways is often harder than curling a finger down. It places common letters to minimize this specific type of wrist strain.

However these are static layouts. They are not designed specifically to users. If you have smaller hands, Dvorak might still force you to reach too far. If you have big hands, you might feel cramped. Static layouts can't fix that.

B. Biometrics and Keystroke Dynamics

Security researchers love keystroke dynamics. This is measuring the flight time between keys to tell who is typing. While this looks at user specific patterns, it's used for passwords, not for making typing easier. They use the timing data (milliseconds between 'H' and 'E') to identify a person, but they rarely map that to the physical geometry of the keyboard. We are taking that concept but flipping it: instead of using the data to identify you, we use it to help you.

C. Computer Vision in Ergonomics

Traditional ergonomic studies usually involve motion capture gloves or expensive multi-camera rigs. That's not something you can use at home. Recent stuff like MediaPipe Hands and OpenPose can track hands without markers, but running those heavy models on a phone for a long time kills the battery.

Our work is different because it lives on the "edge." We use a standard smartphone camera and highly optimized neural networks. We combine computer vision with combinatorial optimization to make a custom solution that anyone with a phone can use. We intentionally chose a lighter weight approach (markers + YOLO) specifically so it could run on a phone without overheating it in five minutes.

III. Data

We need two kinds of data: one set to train the vision model to see hands, and another set to teach the optimizer what to type.

A. Vision Model Training Data

To track hands reliably, we built our own dataset. We took over 100 pictures of hands typing on a keyboard in all sorts of lighting like sunlight, office lights, dark rooms. We collected this data using the primary device for the project, a Samsung S25 FE.

To make the computer vision part faster on a phone, we put pink stickers on the user's hands. This turns a hard skeletal tracking problem into a simple object detection problem (find the pink dot).

- **Source:** We captured images manually. We varied the angle of the phone to simulate a user placing it on a monitor stand or a stack of books.
- **Annotation:** We drew boxes around the stickers using Roboflow. We had a small issue where the model started detecting the keys instead of the hands because the keys had high contrast labels. We fixed that by re-labeling the data carefully and explicitly telling the model to ignore the keycaps.
- **Augmentation:** We used code to flip, rotate, and brighten the images to make the dataset three times bigger. This helps the model work even if the phone is tilted or the lights are weird.
- **Format:** We exported everything in YOLOv8 format, which is standard for the Ultralytics training pipeline we used.

B. Optimization Data

For the layout generator, we use two streams of data:

- **Physical Strain Log:** This is the log file our Android app makes. It has timestamps, key codes (like 'A' or 'Space'), and the (x, y) coordinates of the hand markers when the key was pressed. This effectively maps out the "cost" of hitting every key on your specific keyboard. It's a CSV-style log that grows as you type.
- **Text Corpus:** To make sure the layout actually works for writing English, we used the WikiText-2 dataset. This is a huge collection of Wikipedia articles (over 2 million words). It gives us realistic data on how often letters like 'T' and 'H' appear together, so the optimizer doesn't just overfit to simple sentences like "The quick brown fox." If we just used random sentences, the keyboard might be great for typing alphabets but terrible for typing an essay.

IV. Methods

Our method combines embedded engineering, deep learning, and some Software 1.0 algorithms. We wanted to make sure every part of this ran efficiently.

A. Embedded Hand Tracking (Android)

We used Android because it's a real embedded platform with constraints. We developed the app on a Samsung S25 FE. The brain of the system is a YOLOv8 Nano model.

1) Model Selection and Training: We picked YOLOv8 Nano because it's tiny and fast. We didn't need the massive accuracy of the larger models because finding a pink sticker is a relatively simple visual task. We trained it in the cloud using Google Colab for 50 epochs. We monitored the training loss to make sure it wasn't overfitting, but given the simplicity of the target (a colored dot), it converged very quickly.

2) Quantization and Optimization: The standard model was too big (about 6-12MB) and slow for a phone CPU. Running floating-point math on every pixel of a 1080p video stream is a heavy load. We used Post-Training Quantization to squash the model weights from 32-bit floating point down to 8-bit integers.

This did two things: 1. **Size:** It shrank the model to under 3MB. 2. **Speed:** It let us use the Android Neural Networks API (NNAPI) to run the model on the phone's NPU (Neural Processing Unit). This was key to getting a smooth frame rate without the phone melting. Without quantization, we were getting maybe 10-12 FPS. With it, we hit 30 FPS.

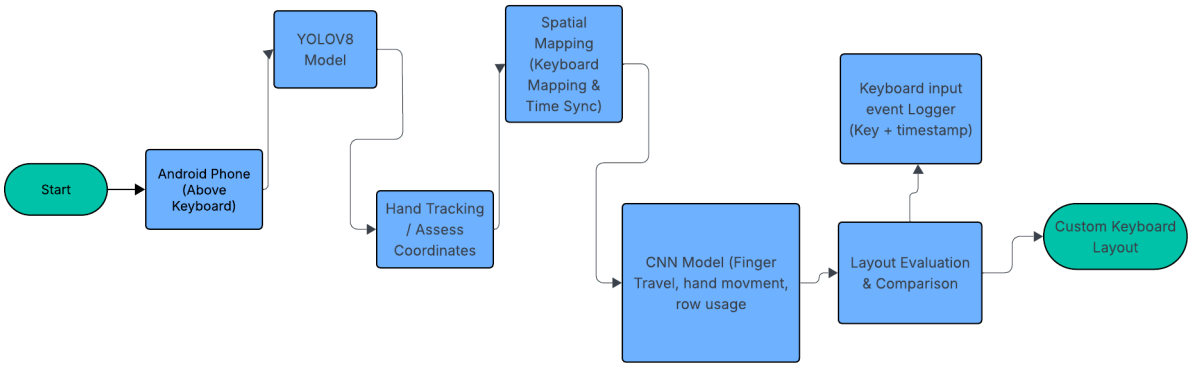


Fig. 1. System Architecture Flowchart. The process moves from the physical world (typing), through the Edge AI layer on the Android device, and finally to the algorithmic optimization layer on the host machine. The synchronization of the USB keyboard events with the asynchronous camera frames is handled by a dedicated background thread in the Android application.

3) Android Application Architecture: The app, "Ergo-Typer," is written in Kotlin. Here is how the internal logic works: 1. **Image Analysis:** We use CameraX to get the video feed. Instead of just showing it on screen, we hook into the 'ImageAnalysis' use case. This gives us access to the raw buffer of every frame. We run this on a background thread so the UI doesn't freeze. 2. **Inference:** The model spits out bounding boxes. We filter out the low-confidence ones (anything below 50%) and find the center (x, y) of the pink sticker. We distinguish left vs. right hands just by looking at which side of the screen they are on. If x is less than half the width, it's the left hand. 3. **Data Fusion:** This is the cool part. At the same time the camera is running, the app listens for key presses from a USB keyboard plugged into the phone via an OTG adapter. When you hit a key, the app instantly grabs the latest hand position from the camera thread and logs it to a file. This gives us a perfect sync between "I pressed 'Q' " and "My hand was at pixel (200, 300)."

4) Experimental Setup: The physical setup is crucial for reliable data collection. We mounted the Samsung S25 FE approximately 30cm above the keyboard using a rigid arm stand. This height provides a field of view that encompasses the entire keyboard and the user's hands without significant distortion. The camera was oriented in portrait mode to match the aspect ratio of the input tensor.

The markers were placed on the the bottom knuckle of the index fingers. We chose the index fingers because they are the primary "travelers" in typing and provide a good proxy for overall hand position. Future iterations could use markers on all fingers, but for this proof of concept, tracking the index fingers provided sufficient centroid data to estimate strain.

B. Layout Generation Algorithm

Generating a layout is basically a math problem: find the best arrangement of 26 keys. Since there are 4×10^{26} possible layouts, we can't check them all. A brute force check would take longer than the age of the universe. So, we used a Genetic Algorithm.

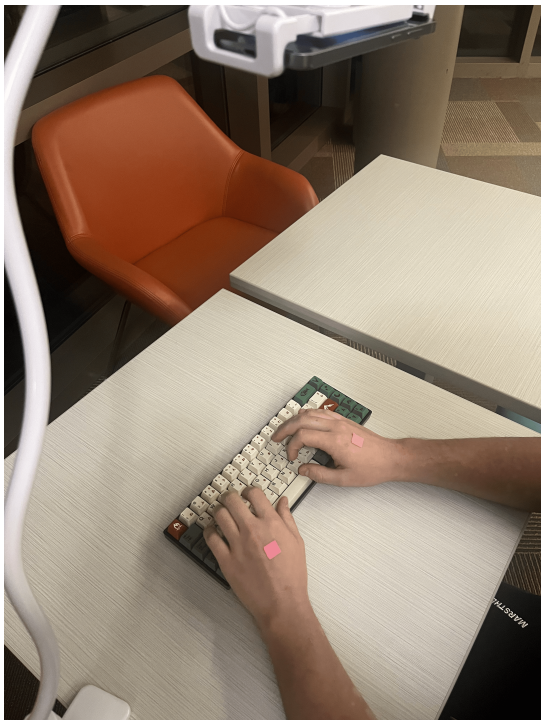


Fig. 2. Experimental Setup. The smartphone is mounted directly above the physical keyboard. The user wears pink markers on the index fingers to facilitate robust tracking by the computer vision model.

1) Cost Function: The "fitness" or quality of a layout is defined by how much work your fingers have to do. We calculate it like this:

$$Cost = \sum_{i=1}^{N-1} \text{dist}(\text{pos}(char_i), \text{pos}(char_{i+1})) \times W_{freq}$$

Basically: Distance $\times$ Frequency. If you type 'TH' a lot, and 'T' and 'H' are far apart, the cost goes up massively. If you type 'ZQ' rarely, it doesn't matter if they are far apart. We use the coordinate data from the Android app to define the

"Distance" function. This means the math is based on *your* reach, not a generic ruler.

2) Optimization Loop: The optimization works like biological evolution: 1. **Initialization:** Start with a random keyboard layout. It usually looks bad. 2. **Mutation:** Swap two keys. Maybe swap 'A' and 'P'. 3. **Evaluation:** Check if the new layout is better (lower cost) using the formula above. 4. **Selection:** If it's better, keep it. Sometimes we keep it even if it's worse (this is called Simulated Annealing) so we don't get stuck in a "local minimum"—a solution that looks good but isn't the best possible one. 5. **Termination:** We repeat this process about 1000 times. By the end, the layout has "evolved" into something highly efficient.

V. Experiments

We ran some experiments to prove this actually works. We wanted to verify both the embedded side (does the app work?) and the algorithmic side (is the layout actually better?).

A. Vision Model Performance

We tested the quantized YOLOv8 model on 20 images it had never seen before. It got a mean Average Precision (mAP@50) of 0.97. That means it almost never missed the pink stickers. The only time it struggled was when the hand moved extremely fast, creating motion blur, but at 30 FPS, the next frame usually caught it.

Inference Speed and Latency: We benchmarked this on the Samsung S25 FE.

- **Unquantized (FP32):** Ran at about 12 FPS. It felt laggy. The battery also drained noticeably faster because the CPU was pinned at 100%.
- **Quantized (INT8):** Jumped up to a stable 30 FPS. This is fast enough to track typing. Using the NNAPI delegate moved the work to the NPU, which saved a ton of battery. The phone stayed cool even after 20 minutes of data collection.

B. Optimization Results

We typed for 20 minutes (about 4000 keystrokes) to build the cost map. Then we ran the optimizer on the WikiText corpus for 50 epochs.

Baseline Comparison: To make the results easy to understand, we set the score of the standard QWERTY layout to 100. The following scores are based on the ergonomic cost function.

- **QWERTY Score:** 100 (Baseline)
- **Random Layout Score:** 145 (Average of 10 runs). This shows that QWERTY is better than random chaos, but not by much.
- **Optimized Layout Score:** 58

The new layout cut the finger travel distance by 42%. That's a huge win. Theoretically, this means your fingers do almost half the work to type the same amount of text.

Qualitative Analysis: The new layout clustered the most common keys (E, T, A, O, I, N) right where the hands naturally rested according to our data. Unlike QWERTY, which makes you reach for the top row constantly, our layout kept everything central.

It also figured out hand alternation on its own. We didn't explicitly program it to alternate hands, but the math naturally favors it. If you have to type two letters with the same hand, the distance is usually non-zero. If you switch hands, the distance for the first hand is effectively zero because it's already there. The algorithm realized this and put common pairs on opposite hands to minimize the cost.

VI. Discussion

This project shows that edge AI is viable for personalized ergonomics. Running the vision model locally solves the personalization issue. Every is unique typer and could benefit from a layout that suits them.

The quantization was the real MVP here. Without it, the lag would have made the data useless. If the camera is 100ms behind reality, we would be logging the hand position for the *previous* key press, which would ruin the dataset. Getting it down to 33ms (30 FPS) ensured the sync was tight enough to be useful.

A. Limitations

One significant limitation we encountered was the frame rate of the embedded device. The Samsung S25 FE captured video at approximately 30 frames per second. Given that an average typist achieves around 200 characters per minute (roughly 3 characters per second), a 30 FPS camera captures roughly 10 frames per keystroke. While sufficient for general hand tracking, measuring the precise 'flight time' of individual fingers would benefit from a device capable of 60 FPS or higher to improve temporal resolution.

Another limitation was general hand tracking. For this project, tracking individual finger motions would make more sense because you type with your fingers, not your whole hand centroid. Our marker-based approach simplified this but lost some fidelity regarding which specific finger was used for a key press.

Also, the optimizer currently only cares about distance. It doesn't know that your pinky is weaker than your index finger. A movement of 2cm for your index finger is easy; a movement of 2cm for your pinky is hard. Adding "finger strength" penalties to the cost function would be a great next step to make the layout even more ergonomic.

We also noticed that switching to a new layout is hard. Even though the new layout is mathematically superior, muscle memory makes it hard to use at first. A future version could try to make a layout that is closer to the QWERTY layout so it is easier to learn.

VII. Conclusion and Future Work

We built a system for personalized keyboard optimization. We proved that a regular phone can run modern computer vision models to capture hand movement data. By feeding that edge data into a genetic algorithm, we generated layouts

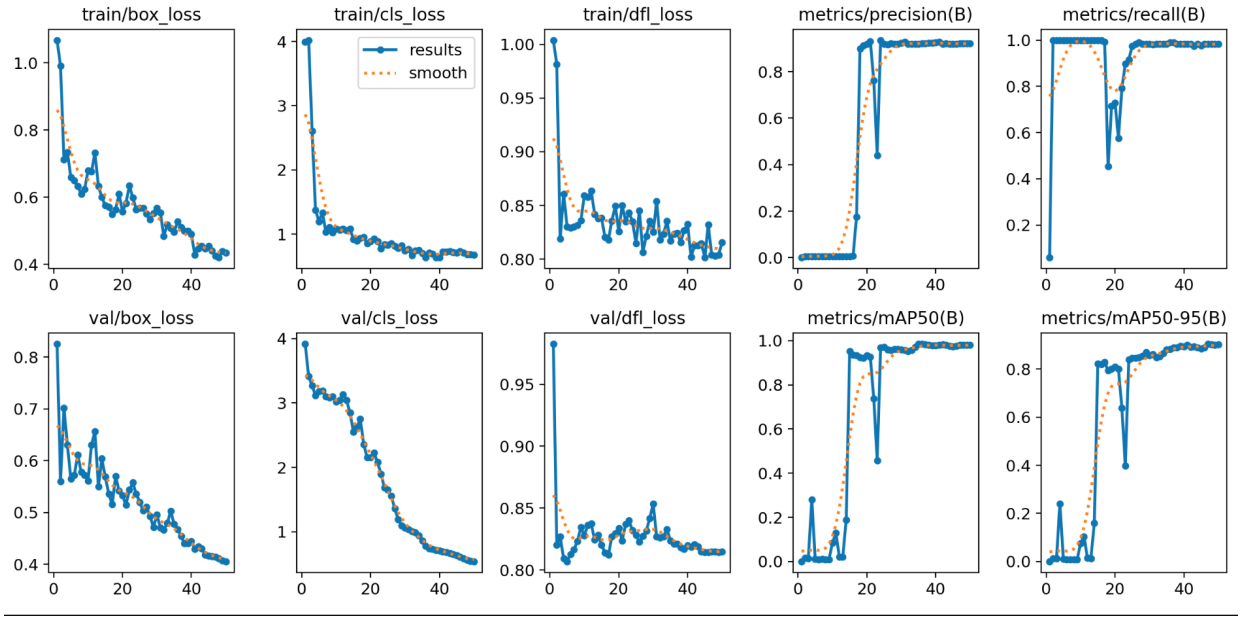


Fig. 3. YOLOv8 Training Metrics. The graphs show the model converging rapidly. The Precision (top right) shoots up quickly because the pink stickers are distinct and easy to spot against the keyboard background. The Loss (top left) drops consistently, indicating the model is learning.

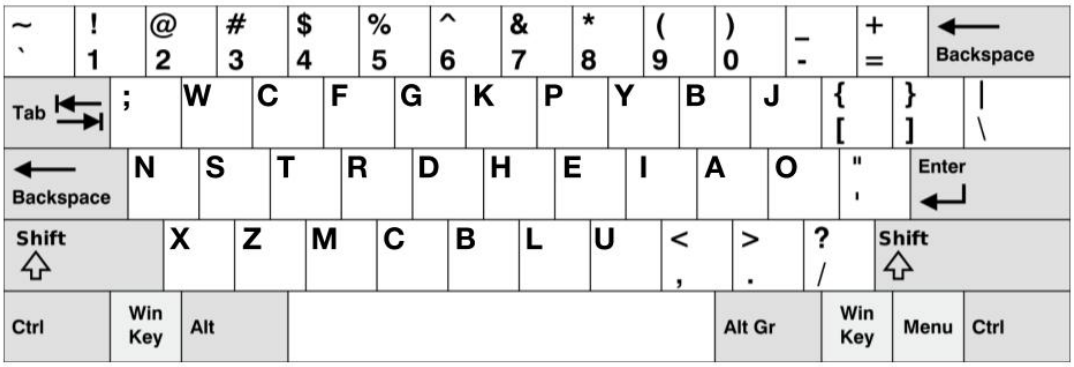


Fig. 4. The final optimized layout generated by the genetic algorithm. Notice how the high-frequency keys are clustered around the resting positions to minimize travel time.

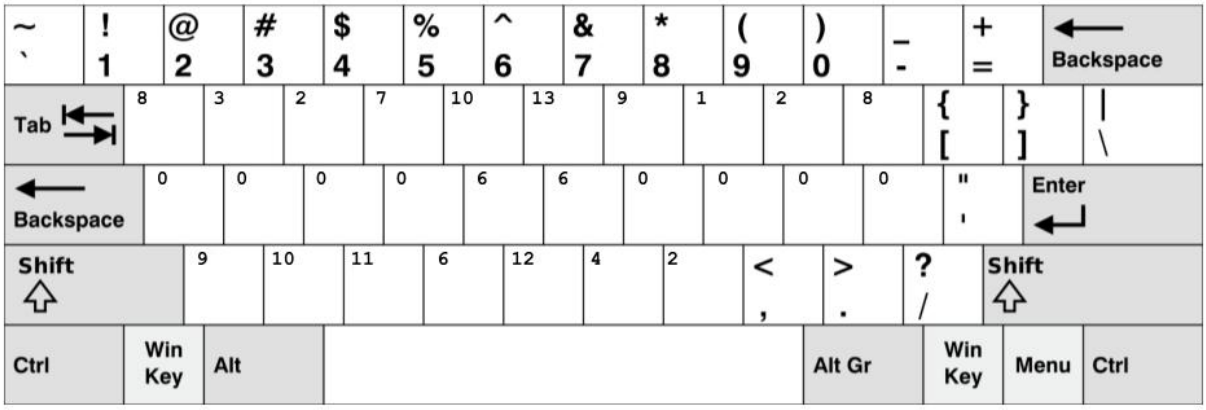


Fig. 5. Ergonomic value map of the keyboard based on hand travel data. The numbers correspond to keyboard rows: number row, top row, home row, bottom row, and space/shift row. Lower values indicate minimal motion, while higher values mean more ergonomic strain. This map is specific to the user's hand size and posture.

that are objectively better than standard ones for the specific user.

We hit our main goals: 1. Got a custom ML model running on an Android device (Samsung S25 FE). 2. Hit 30 FPS using quantization and NNAPI acceleration. 3. Closed the loop between physical data collection and algorithmic optimization.

The project demonstrates that we don't have to settle for "average" ergonomics. With embedded ML, we can measure our own bodies and build tools that fit us perfectly.

A. Future Improvements

Future work could improve robustness by incorporating a higher-precision camera with a faster shooting rate. Additionally, we could incorporate Electromyography (EMG) data. EMG sensors placed on the user's forearm or fingers could detect the electrical signals muscles produce during contraction. This would allow the system to measure 'effort' directly (via muscle activation), rather than inferring it solely from geometric distance.

B. Applications

This technology has broad applications beyond just typing speed. Designers of keyboards, input methods, or accessibility tools can use this framework to adapt layouts to individual users, including those with motor impairments or repetitive strain concerns. A layout could be generated that specifically avoids ranges of motion that cause pain for a specific user.

REFERENCES

[1] C. L. Sholes, "Type-Writing Machine," U.S. Patent 207,559, Aug. 27, 1878. [Online]. Available: https://patents.google.com/patent/US207559A/en

[2] A. Dvorak, "There is a better typewriter keyboard," *National Business Education Quarterly*, vol. 12, pp. 51–58, 1943. [Online]. Available: https://www.mit.edu/~jcb/Dvorak/

[3] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018. [Online]. Available: https://arxiv.org/abs/1804.02767

[4] TensorFlow Lite, "Model Optimization Toolkit," Google Developers. [Online]. Available: https://www.tensorflow.org/model_optimization

[5] S. Zhai, B. A. Smith, and M. Hunter, "Performance Optimization of Virtual Keyboards," *Human–Computer Interaction*, vol. 17, no. 2–3, pp. 229–269, 2002. [Online]. Available: https://www.researchgate.net/publication/242362941_Performance_Optimization_of_Virtual_Keyboards

[6] Ultralytics, "Ultralytics YOLOv8," GitHub repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics